Desenvolvimento de Microserviços

Prof. Guilherme Alves

FACULDADE Donaduzzi

DDD (*Domain-Driven Design*)

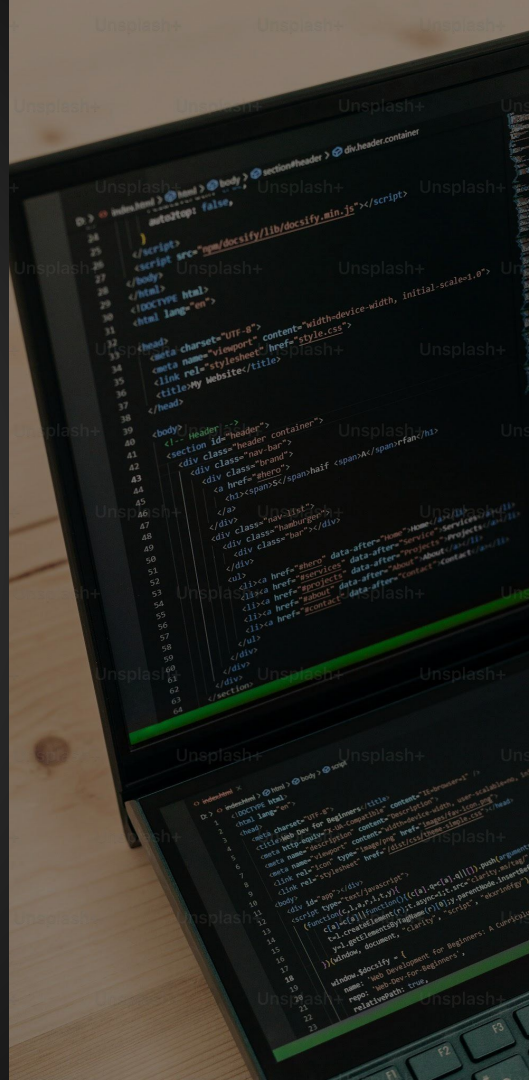
Colocar o domínio do negócio no centro do desenvolvimento do software.

Complexidade

Complexidade é a quantidade de estados possíveis e interações entre os elementos de um sistema.

A complexidade de software é dividida em duas categorias:

- Complexidade Essencial
- Complexidade Acidental



Complexidade essencial

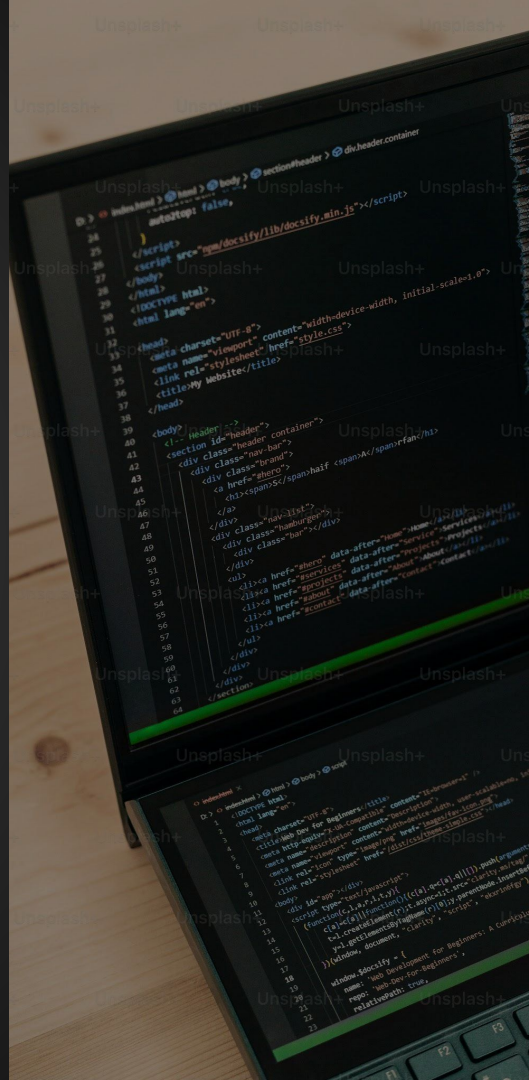
É a complexidade que não pode ser removida.

Um sistema bancário, por exemplo, precisa lidar com:

- Juros compostos
- Risco de crédito
- Inadimplência
- Regras regulatórias

Isso não é complicação técnica, é complexidade do domínio.

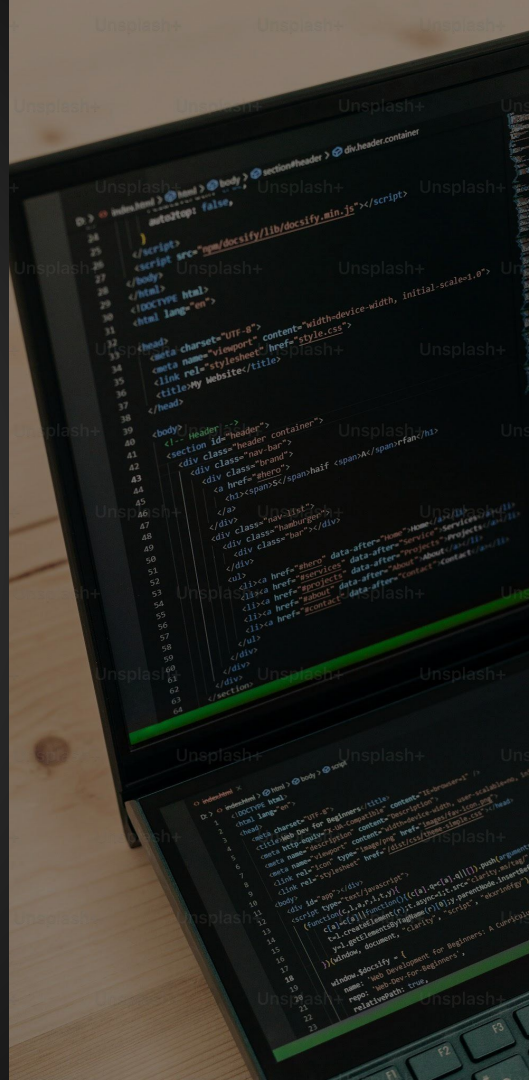
O **DDD** é uma resposta direta à **complexidade essencial**.



O que é e o que não é...

- DDD não é framework.
- Não é arquitetura pronta.

É uma abordagem **estratégica** e **tática** para modelar sistemas complexos.



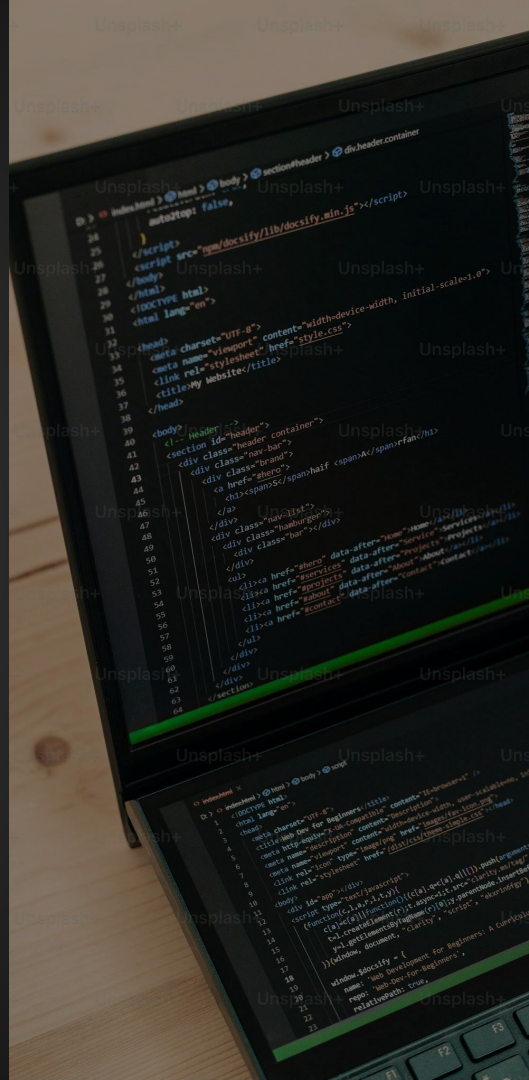
Problema que o DDD resolve

Em sistemas complexos normalmente ocorre:

- Código orientado a banco de dados
- Regra de negócio espalhada
- Linguagem técnica diferente da linguagem do negócio
- Modelo anêmico (*Anemic Domain Model*)
- Forte acoplamento

DDD propõe:

- Modelo rico (*Rich Domain Model*)
- Linguagem compartilhada
- Limites bem definidos
- Foco no que é crítico para o negócio



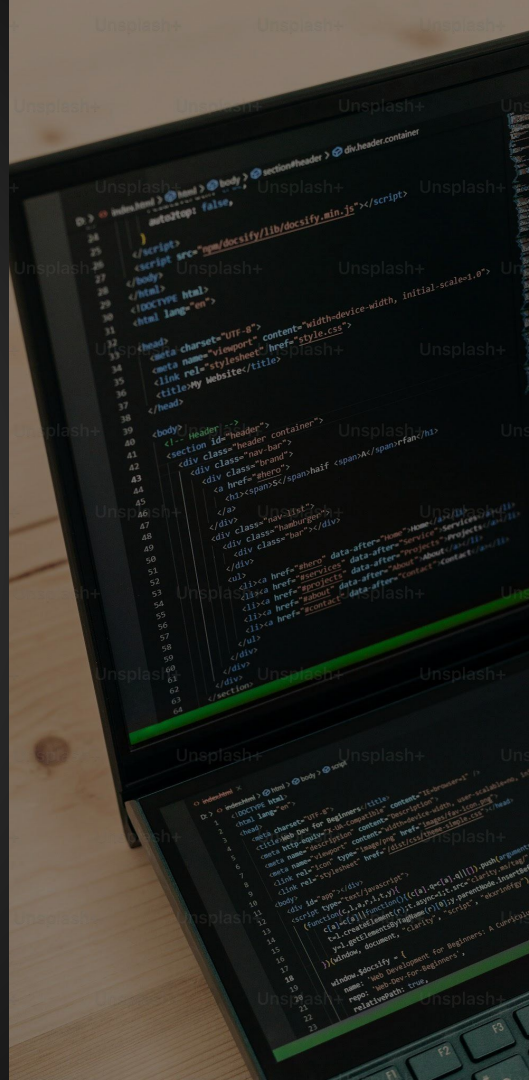
Mudança de mentalidade

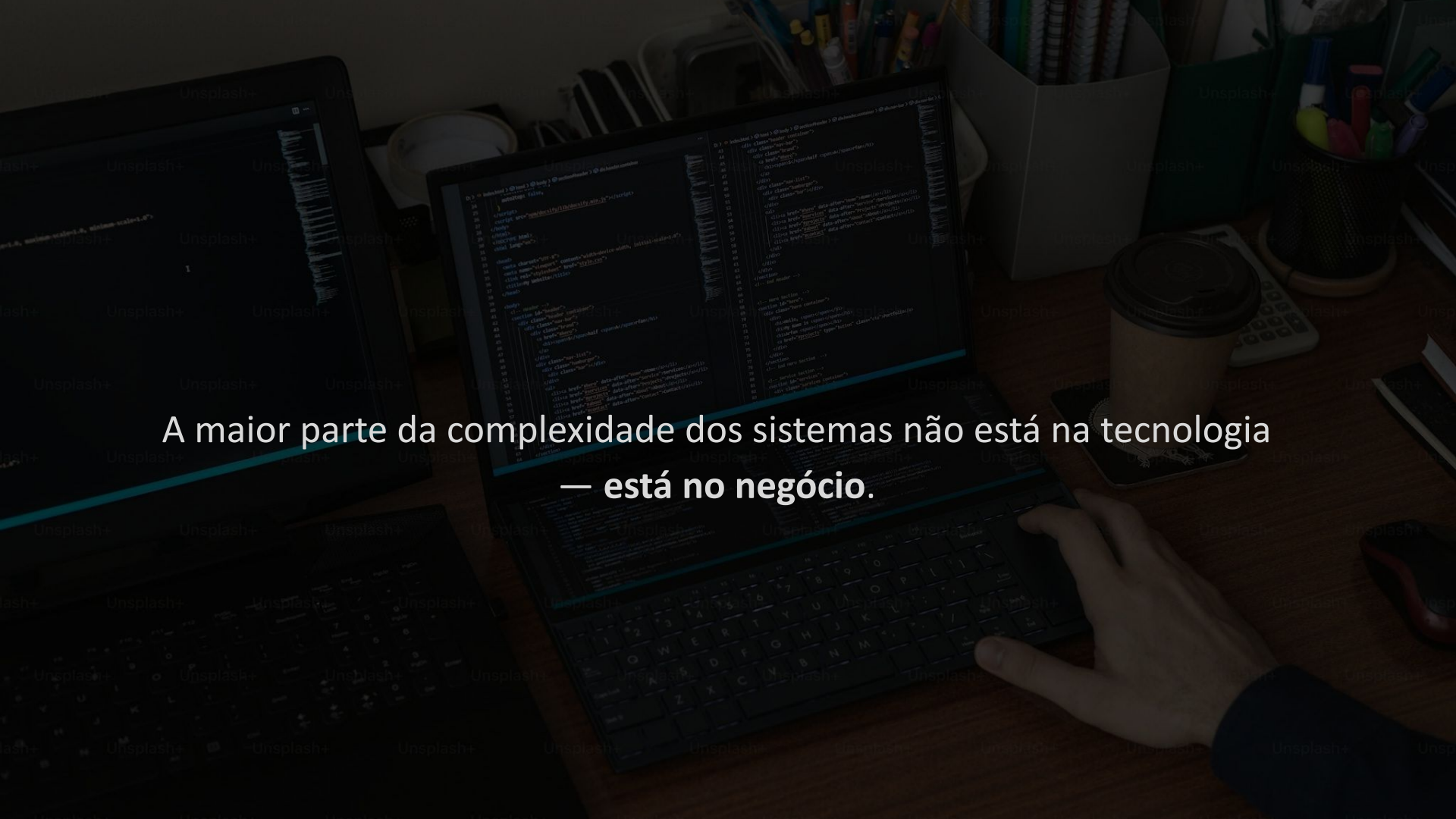
DDD exige três mudanças fortes:

- Código como comunicação
- Modelar comportamentos, não só dados
- Foco no negócio, não no CRUD

Pergunta deixa de ser: Como salvo no banco?

Passa a ser: O que significa cancelar um pedido?



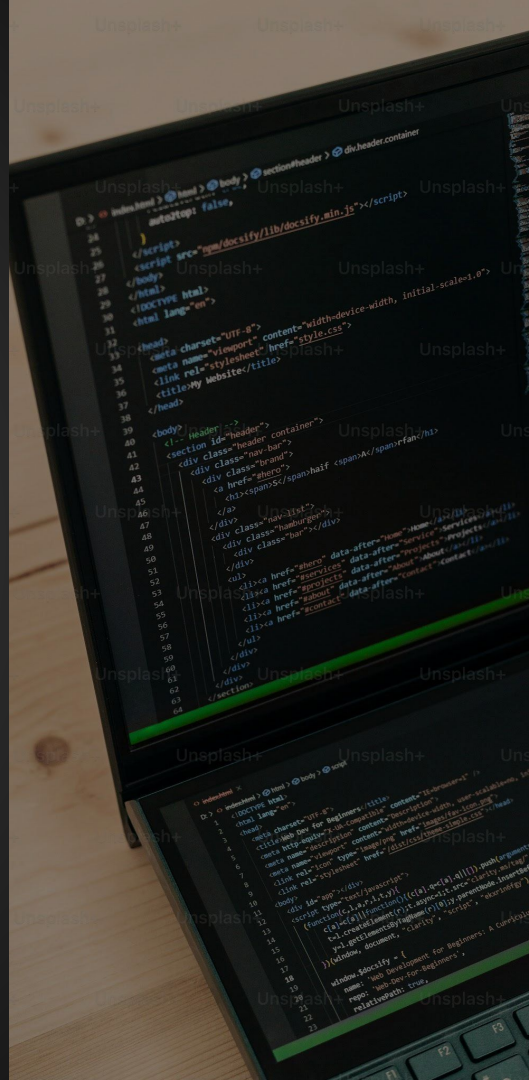


A maior parte da complexidade dos sistemas não está na tecnologia — **está no negócio**.

DDD Estratégico

O **DDD** estratégico é justamente a camada que organiza o nível organizacional e arquitetural, antes do código.

- Como dividir o sistema em contextos bem definidos
- Como esses contextos se relacionam
- Como alinhar software com estratégia de negócio
- Como evitar caos semântico em sistemas grandes



Domínio

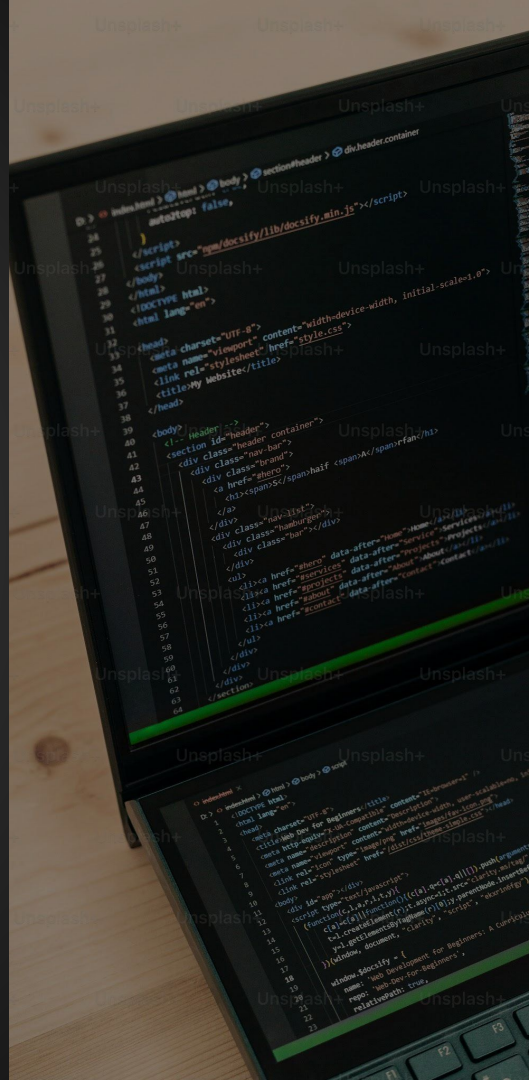
O conjunto de conhecimentos, regras, conceitos, processos e problemas de negócio que o software precisa resolver.

Domínio = o problema que o software resolve.

Exemplos:

- **Sistema bancário** → domínio financeiro
- **Sistema logístico** → domínio de transporte e entrega
- **E-commerce** → domínio de vendas, pagamento, estoque

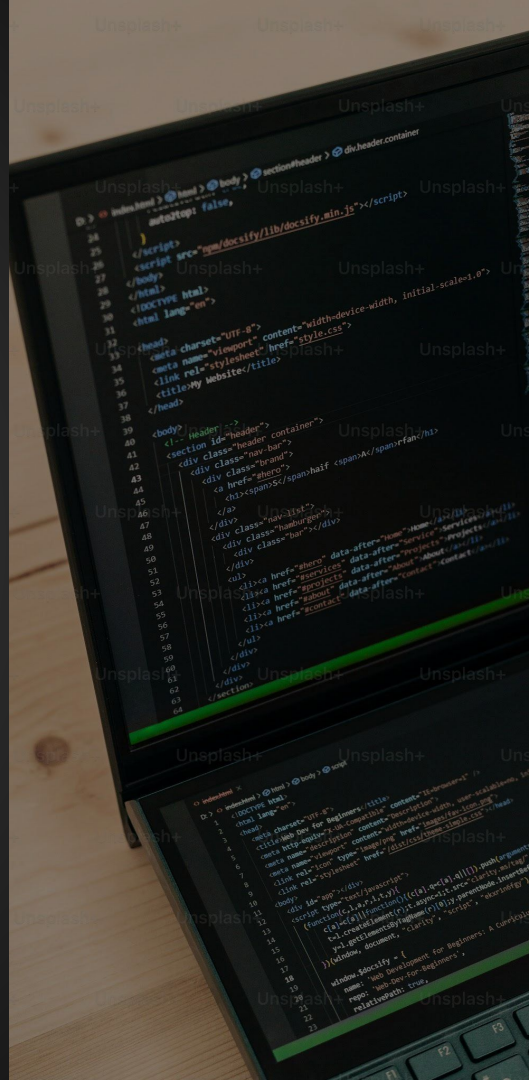
Obs. Domínios grandes são divididos em partes menores chamadas de **subdomínios**.



Domínio

DDD defende que os desenvolvedores devem ter um profundo conhecimento do domínio do sistema que eles desenvolvem. Esse conhecimento deve ser obtido por meio de conversas e discussões frequentes com especialistas no domínio (ou no negócio).

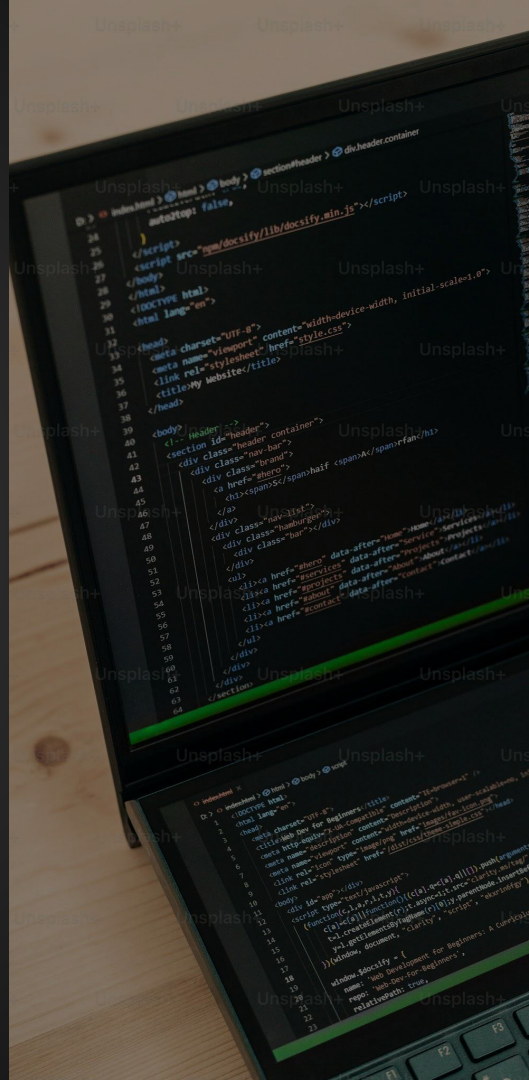
Portanto, o design do sistema deve ser norteado para atender ao seu domínio. E não, por exemplo, para se moldar a uma determinada tecnologia de programação.



Conhecimento do Domínio

Você precisa internalizar:

- Regras
- Exceções
- Fluxos
- Restrições
- Conceitos implícitos



Linguagem Ubíqua

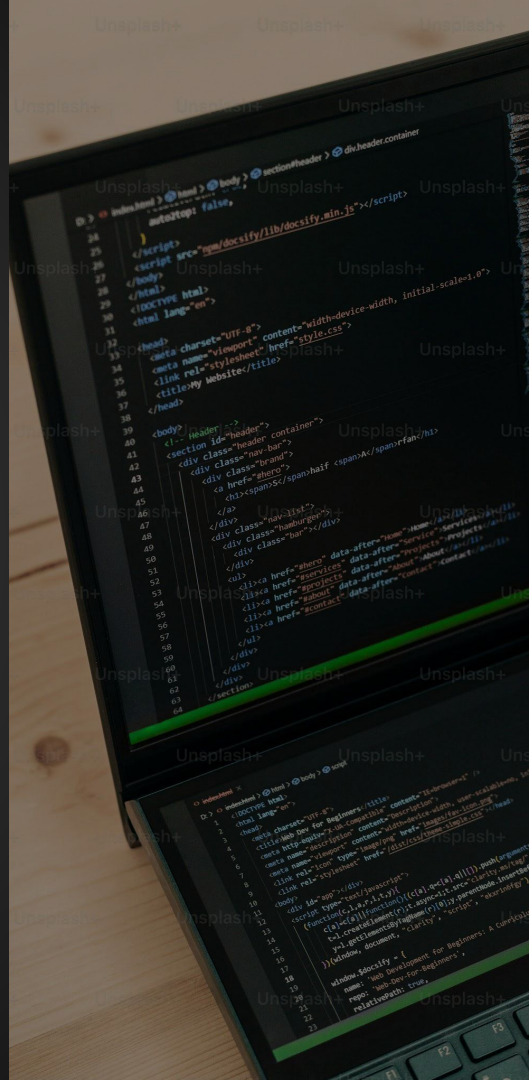
É a linguagem comum, compartilhada entre especialistas de domínio e desenvolvedores, usada tanto nas conversas quanto no código.

O maior problema em sistemas complexos não é código, é comunicação ambígua.

Se o negócio fala “**Pedido**”, o código fala “**Pedido**”.

Um especialista diz: “*Um pedido pode ser cancelado se ainda não foi faturado.*”

O time técnico traduz para: *order.cancelIfNotInvoiced()*



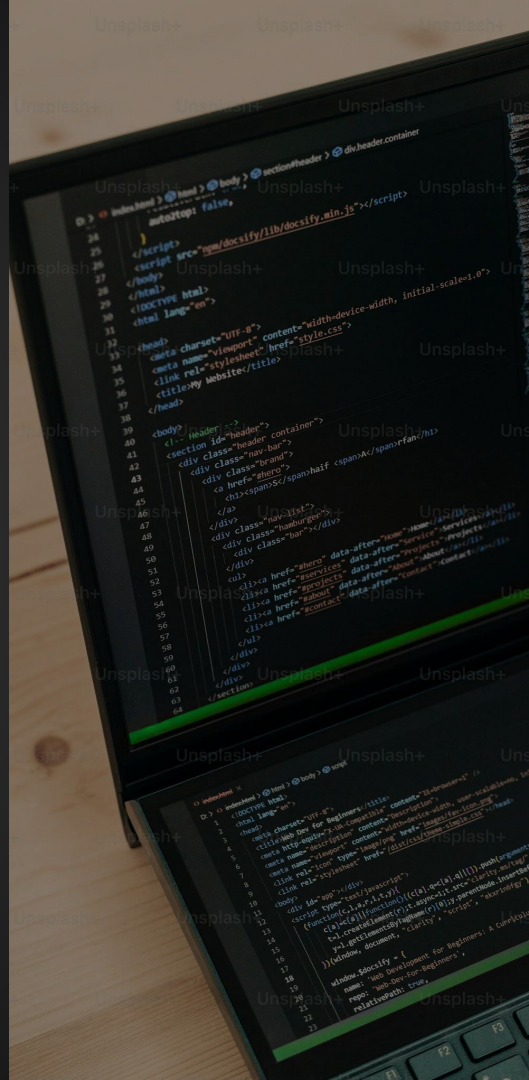
Bounded Context

Bounded Context é um limite explícito dentro do qual um modelo de domínio é válido e consistente.

É um recorte delimitado dentro do domínio onde um modelo específico é válido e consistente.

Tem sua própria linguagem ubíqua

- Possui regras próprias
- Tem fronteiras bem definidas
- **Pode** ter banco de dados próprio
- **Pode** ser um microserviço



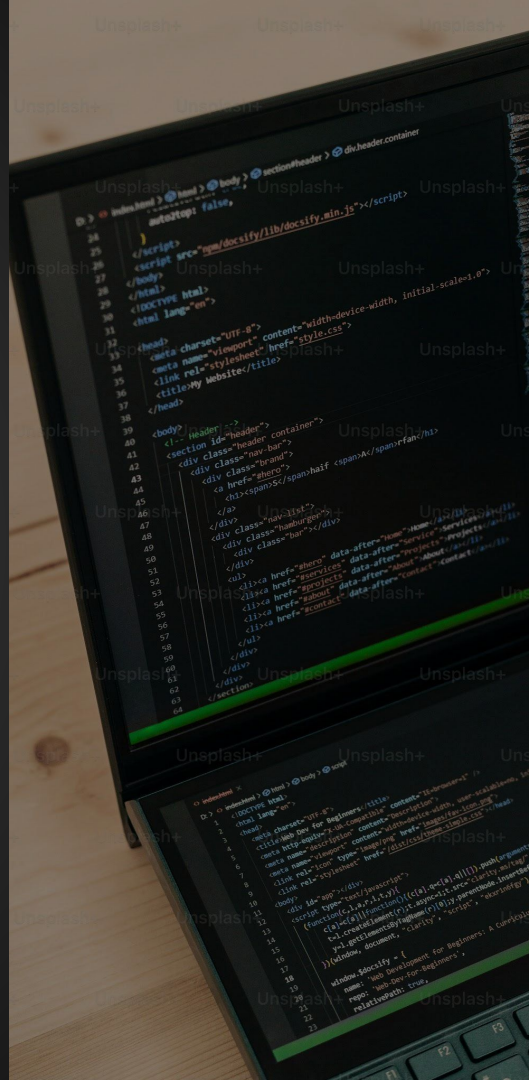
Context Mapping

Ele define os relacionamentos entre contextos.

Se Bounded Context é o limite semântico, o Context Map é o mapa político e técnico entre esses limites.

Ele responde:

- Quem depende de quem?
- Quem dita o modelo?
- Onde precisa de tradução?
- Onde há risco de acoplamento?
- Onde está o poder?



Domínio → O que estamos tentando resolver

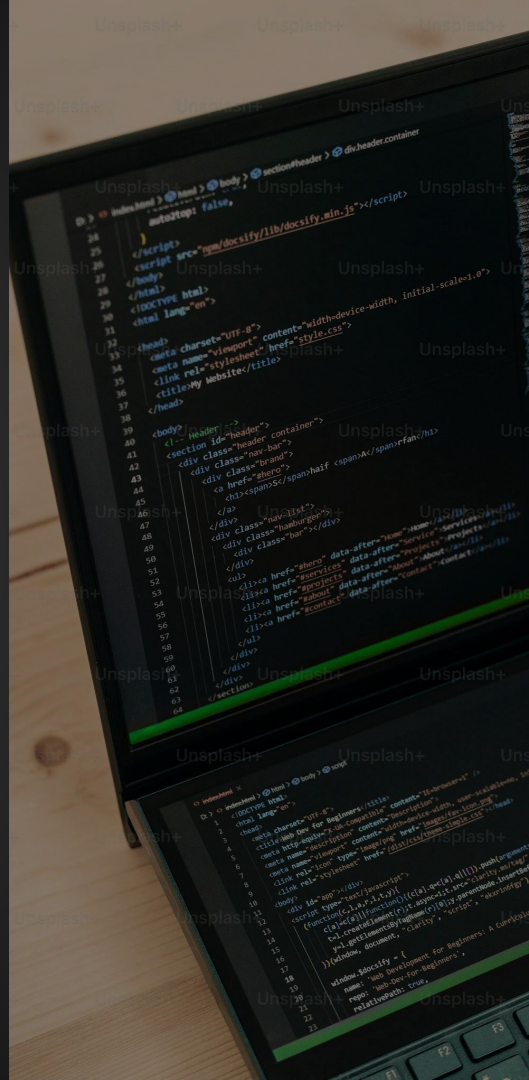
Linguagem Ubíqua → Como falamos sobre o domínio

Bounded Context → Onde essa linguagem é válida

Context Mapping → Como eles se comunicam

Criação de um Design Estratégico

1. Definição dos Domínios
2. Definição dos Bounded Contexts
   a. Linguagem Ubíqua
3. Definição dos Context Mapping

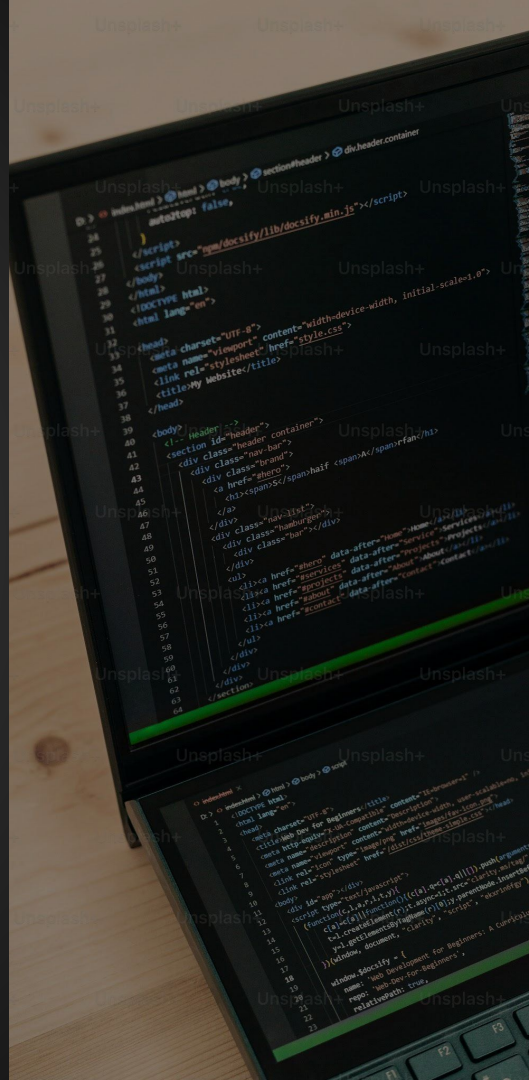


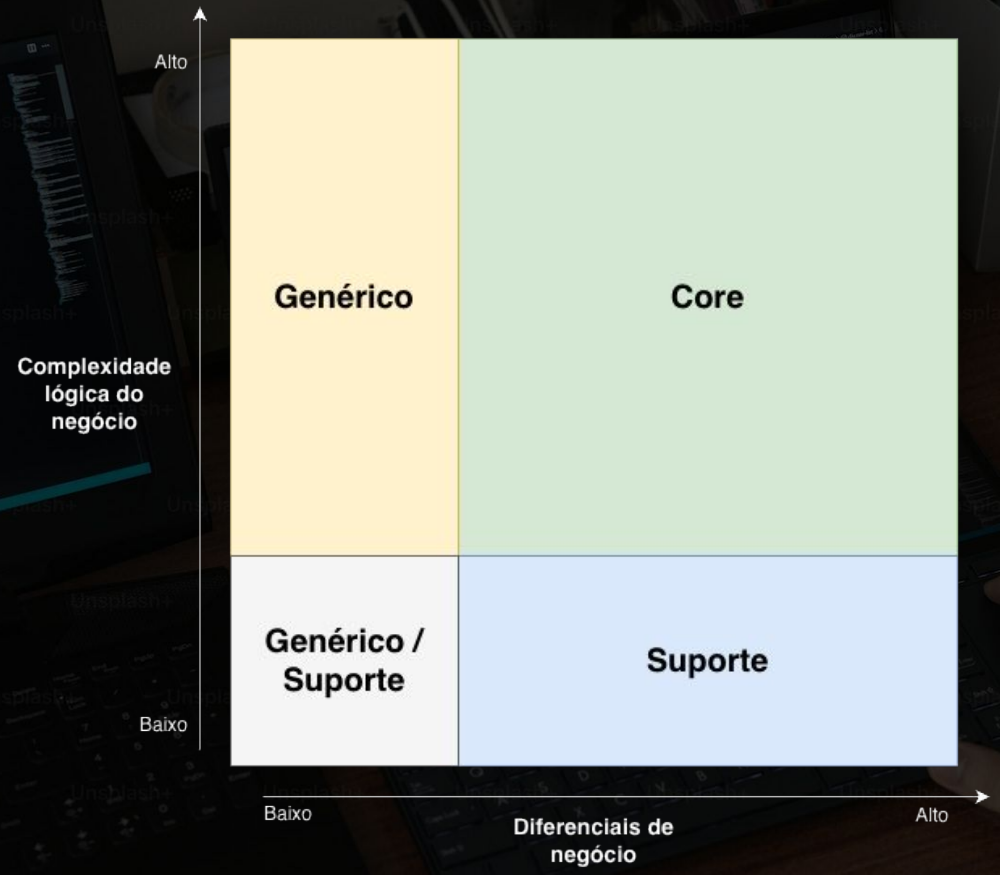
Definição dos Domínios

Core Domain - (valor estratégico alto) onde está o diferencial competitivo e a maior parte da complexidade essencial.

Supporting Domain - (valor estratégico médio) é um subdomínio que apoia o Core Domain (é específico da empresa), possui regras e particularidades do negócio, mas não constitui o diferencial competitivo da organização.

Generic Domain - (valor estratégico baixo) é um subdomínio comum ao mercado, não específico da empresa, que não gera vantagem competitiva e cuja solução pode ser adquirida ou reutilizada sem prejuízo estratégico.

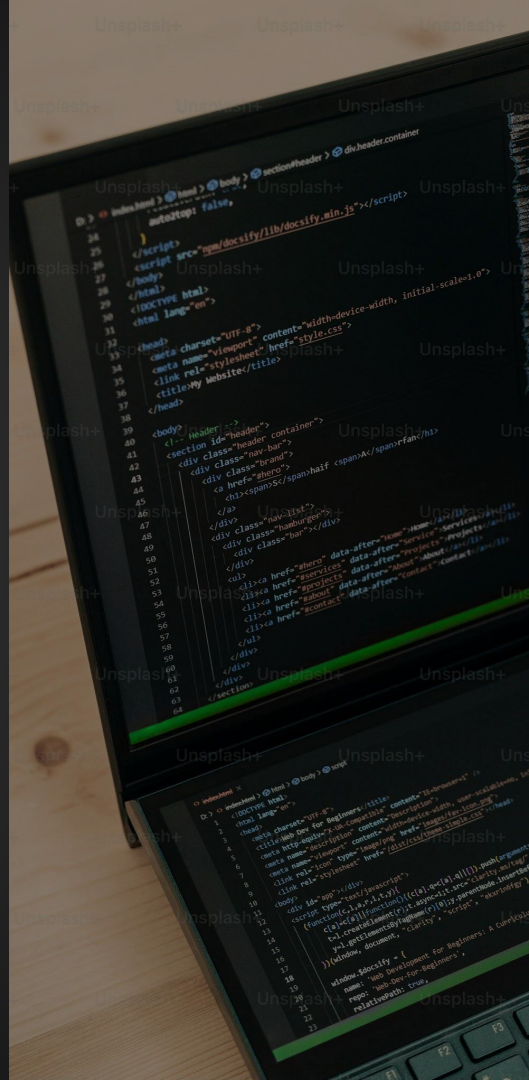



Alto
Complexidade lógica do negócio
Baixo
Genérico
Core
Genérico / Suporte
Suporte
Baixo
Diferenciais de negócio
Alto

Definição dos Bounded Contexts

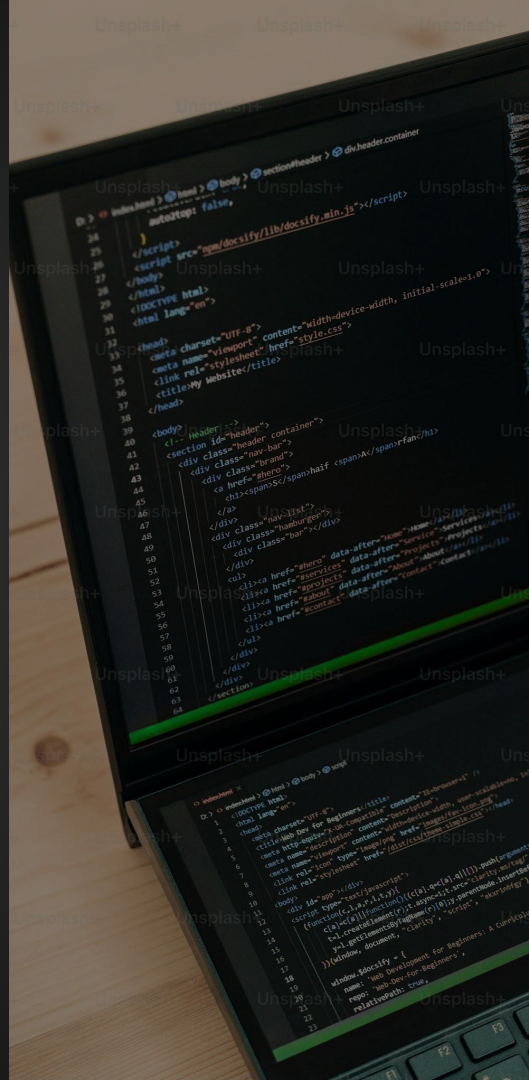
Cada estado pode ser um **bounded context**, contendo espaço delimitado, com regras específicas e com sua própria **ubiquitous language**.

Neste caso o país seria um domínio.



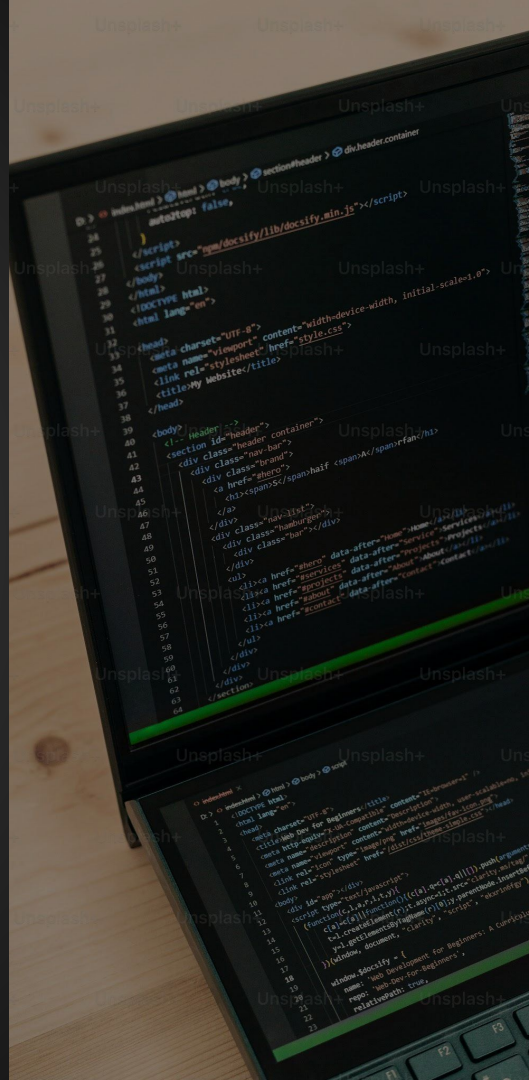
Definição dos Bounded Contexts

1. Explicitamente defina o contexto dentro do qual um modelo se aplica.
2. Limite as fronteiras em termo de organização, uso da aplicação ou até física em termo de código e esquema de banco de dados.
3. Mantenha o modelo estritamente consistente dentro desses limites.
4. Faço a definição do **glossário** que será utilizado como linguagem.

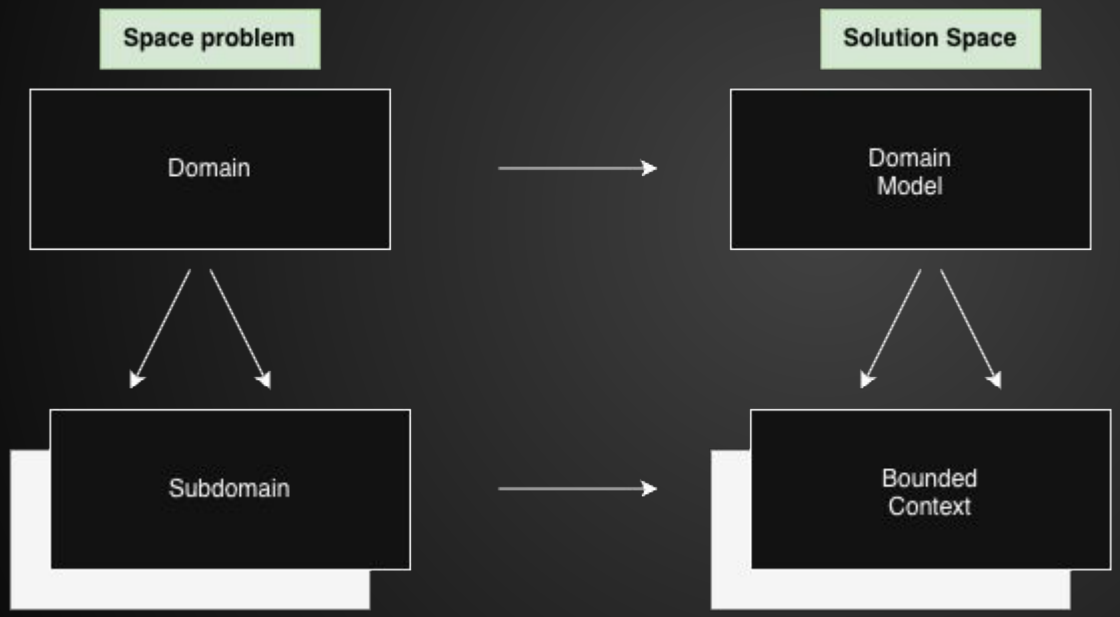


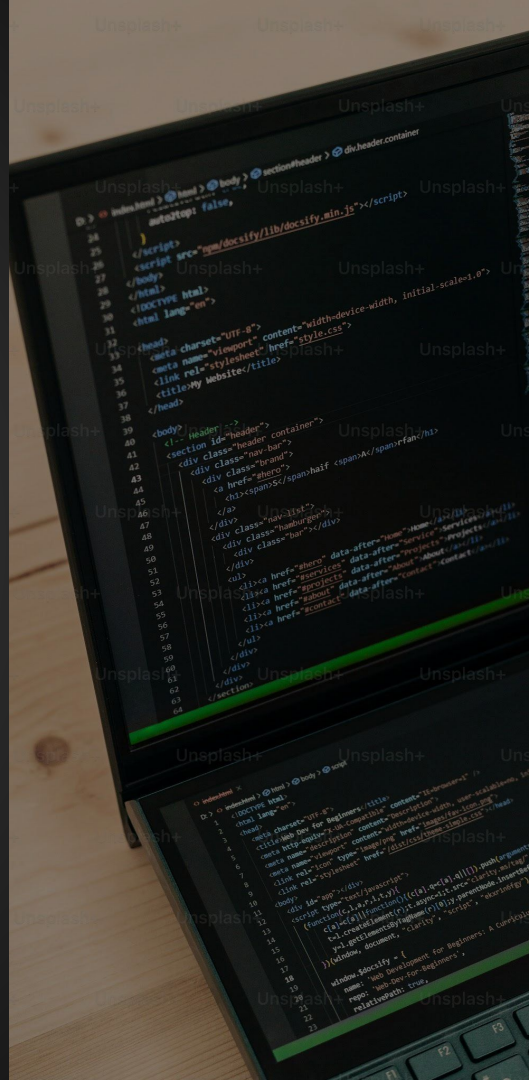
Definição dos Bounded Contexts

Na teoria tudo isso é muito lindo, mas na prática…



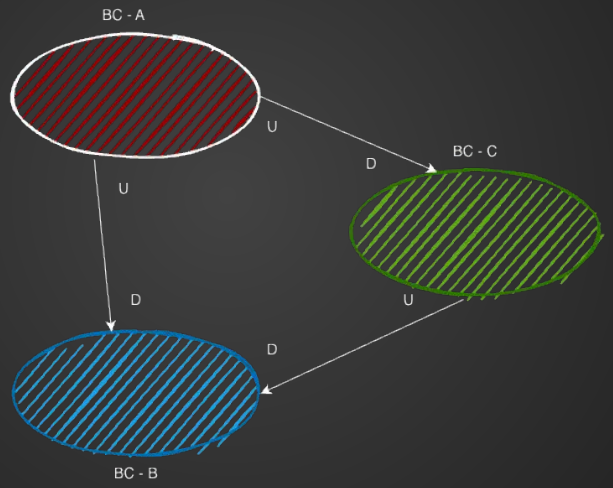
Definição dos Bounded Contexts

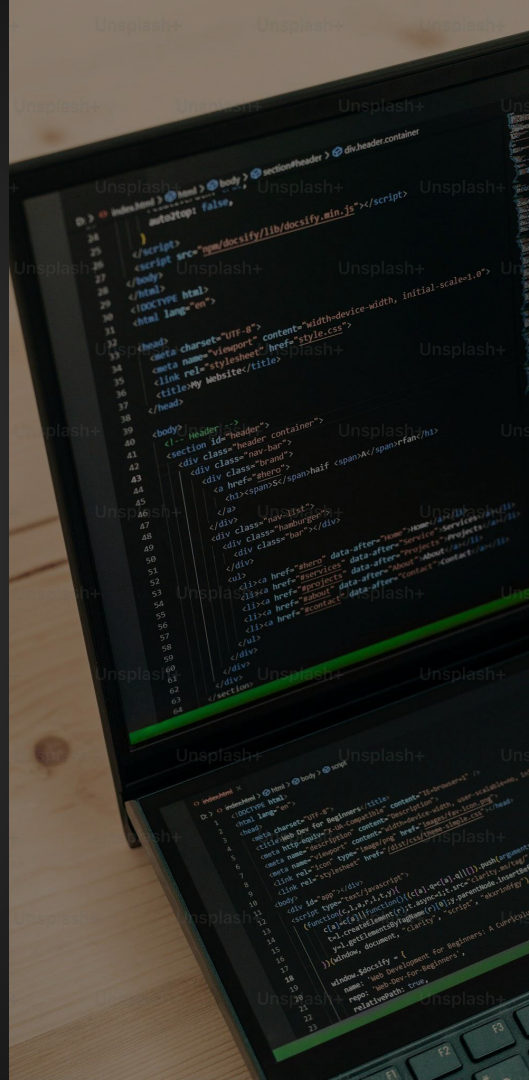




Definição do Context Mapping

Voltando para o exemplo do mapa do país, o **context mapping** seria análogo a imagem disposta, mostrando como os contextos se relacionam.





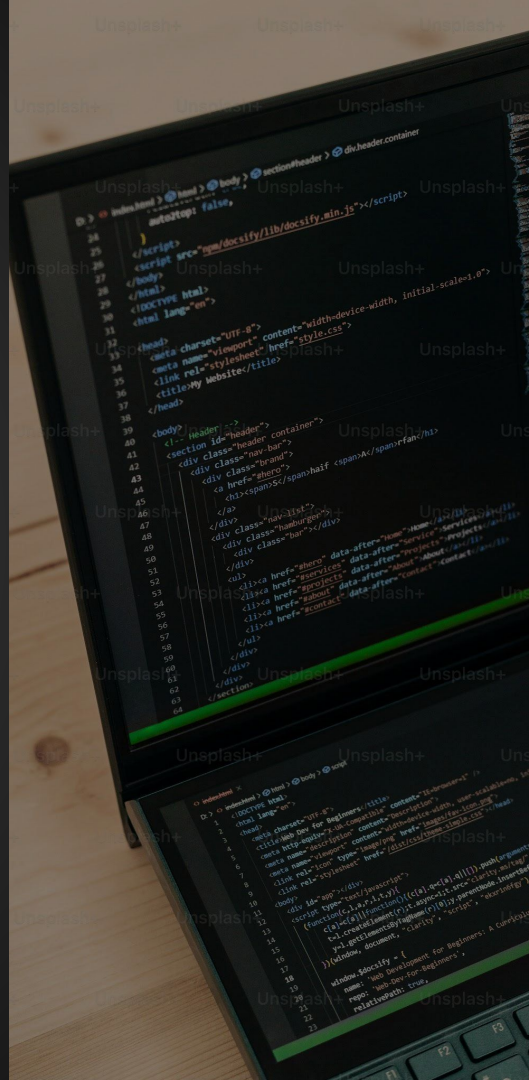
Definição do Context Mapping

Outra coisa muito importante, é identificar como é o relacionamento entre os bounded contexts do domínio. Posso classificar em:

U - *Upstream* - O contexto que fornece algo

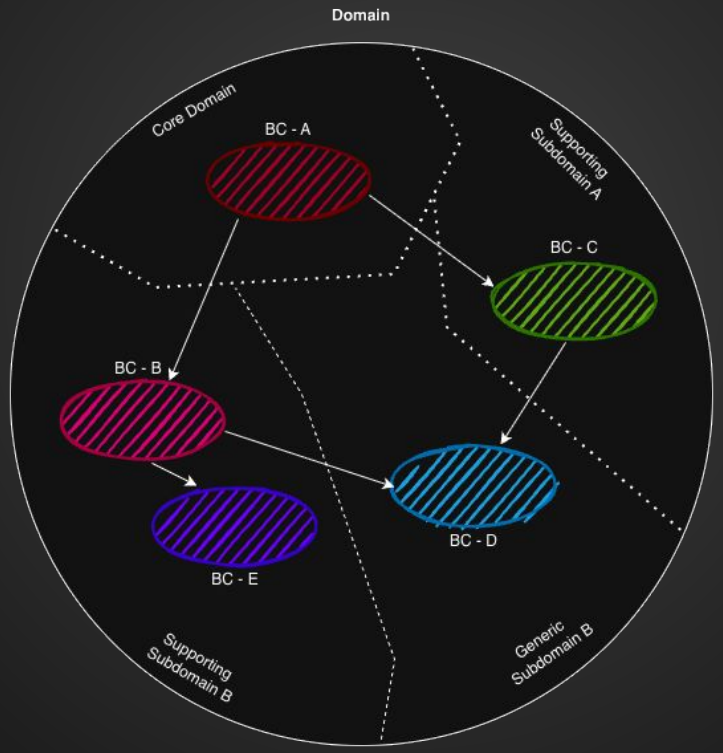
D - *Downstream* - O contexto que depende de outro

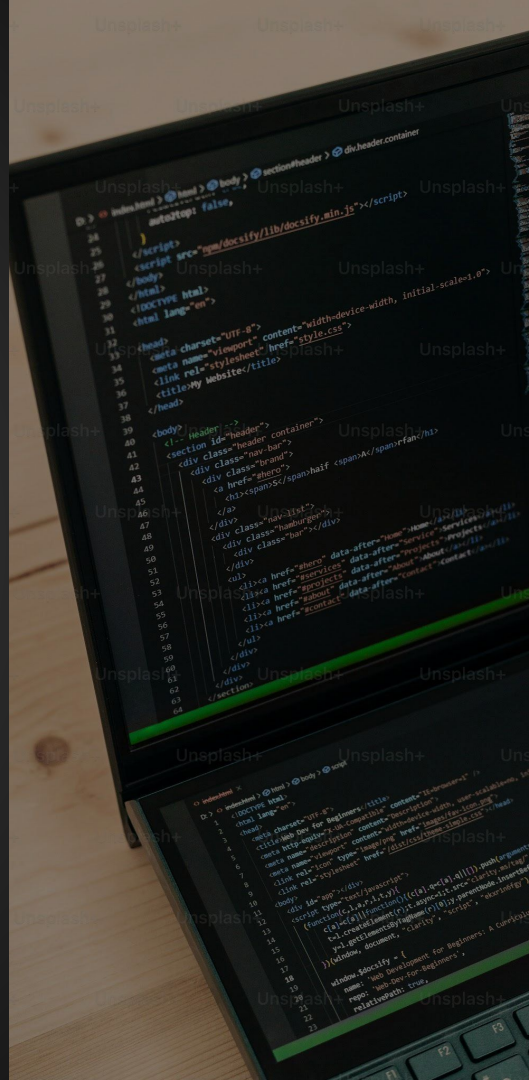
Em resumo, eu consigo saber onde uma alteração pode vir a impactar.



Definição do Context Mapping

No context mapping também faço o agrupamento dos bounded context.





Imagine o seguinte problema…

A empresa tem como negócio principal a fabricação de tintas automotivas e está espalhada pelo mundo todo. Como qualquer empresa, muitas decisões são feitas em sua sede.

O problema é que em todas as empresas filiadas, espalhadas pelo mundo, necessitam de manutenções prediais. Seja pintura, jardinagem, elétrica…

Chegada do problema…

O analista de negócio, no meio de todas as suas demandas e prazos curtos, chega pedindo uma solução de manutenção predial para empresas filiadas para que uma contratada possa efetuar o serviço.

Mais detalhes do problema…

Um analista de patrimônio faz um pedido de manutenção informando o motivo, qual a filiada e o contrato. O contrato é feito com uma contratada local.

O contratado é buscado no ERP onde são guardados todos os contratos da empresa, seja sede ou filial.

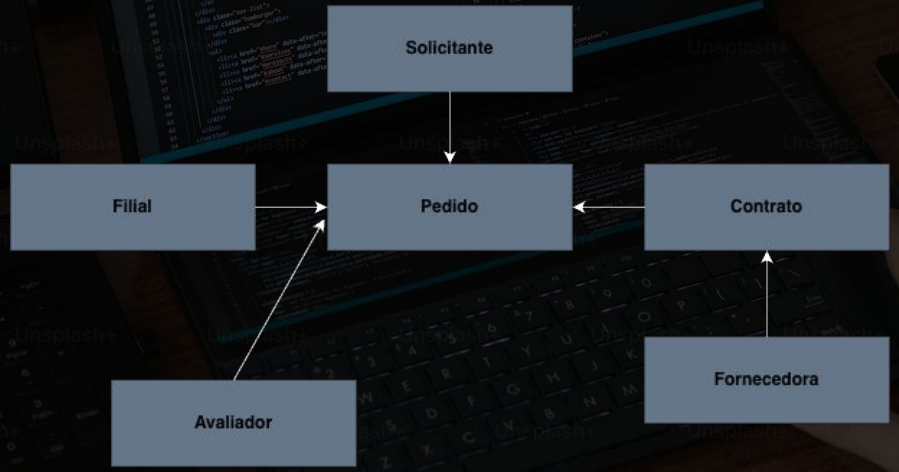
Cada pedido deve ser enviado para o diretor da empresa que fica na sede.

Mais detalhes do problema…

O avaliador valida o pedido e em caso de não aprovação o mesmo deve informar o motivo.

Após a aprovação do pedido, o gestor da filial local, onde a manutenção será feita, deverá informar a data de finalização do pedido para efetuar o pagamento da prestadora.

Como resolvemos esse problema, visando que não é algo tão complexo...



ISSO NÃO É DDD!

Domain Experts…

Em conversa com o analista, foi informado que a **solicitação** é realizada informando o número do contrato e a terceirizada, além da justificativa, para efetuar a manutenção na subsidiária.

A solicitação é enviada para aprovação do **aprovador** que tem a opção de reprovar com uma **justificativa**.

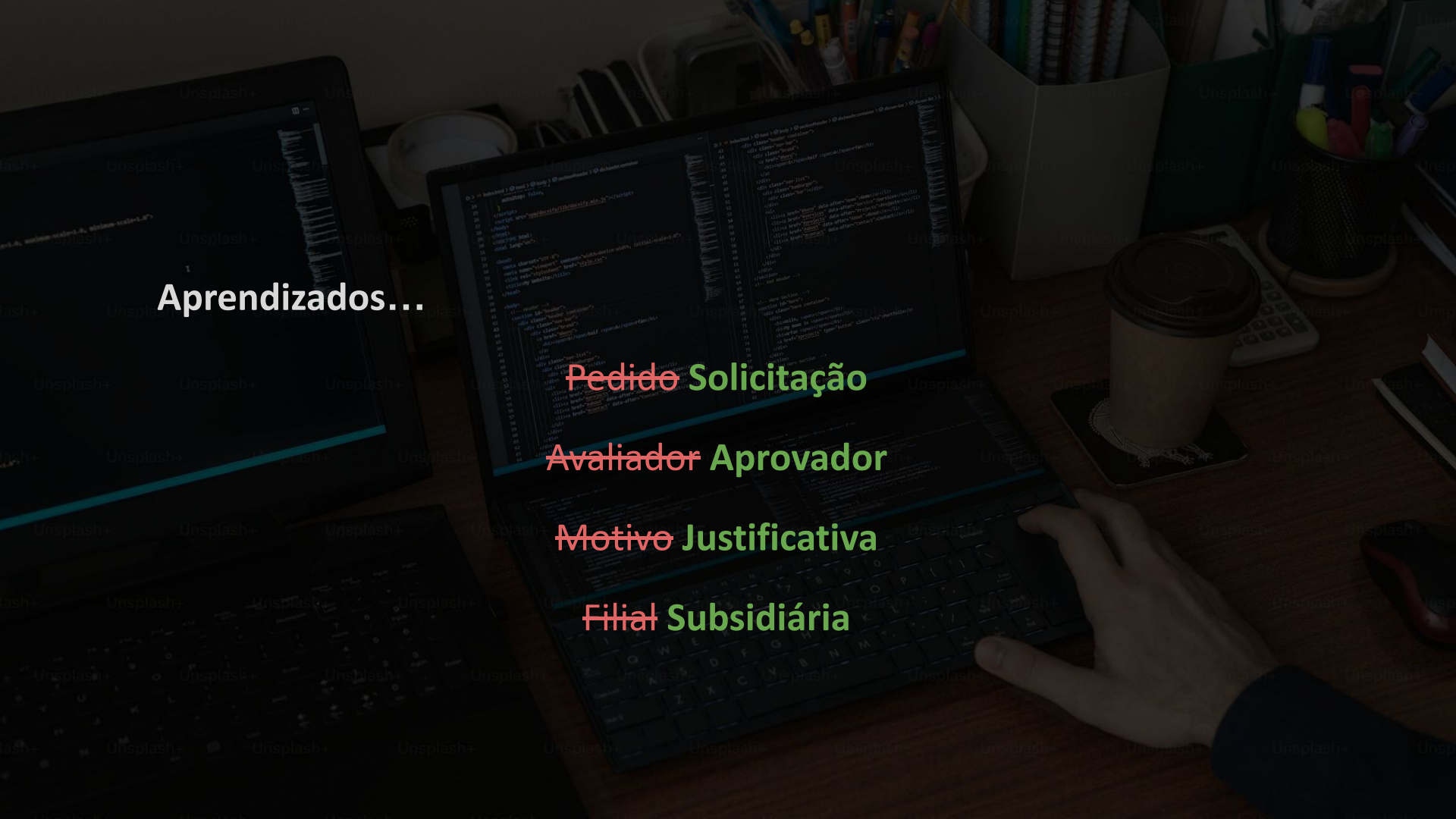
O contrato é **buscado no ERP Contracts**, onde também é possível acessar a terceirizada.

Domain Experts…

Em conversa com o diretor geral, foi informado que ele recebe a solicitação e analisa com base na **vigência do contrato** e a **justificativa** do solicitante para aprovação ou reprovação da mesma.

Em caso de dúvida sobre o contrato, acessa o gestor de contrato que está na solicitação.

Após a aprovação, envia a solicitação para o gestor da subsidiária que irá receber a manutenção para fiscalização e finalização do serviço.



Aprendizados…

~~Pedido~~ Solicitação

~~Avaliador~~ Aprovador

~~Motivo~~ Justificativa

~~Filial~~ Subsidiária

Domain Experts…

Em conversa com a gestor local, uma vez que o **serviço** para manutenção é liberado, é informado aos **terceirizados** da **empresa contratada** para que tenham as devidas permissões de entrada.

Após a finalização do serviço, o mesmo é **encerrado** para que a contratada possa receber pelo serviço.

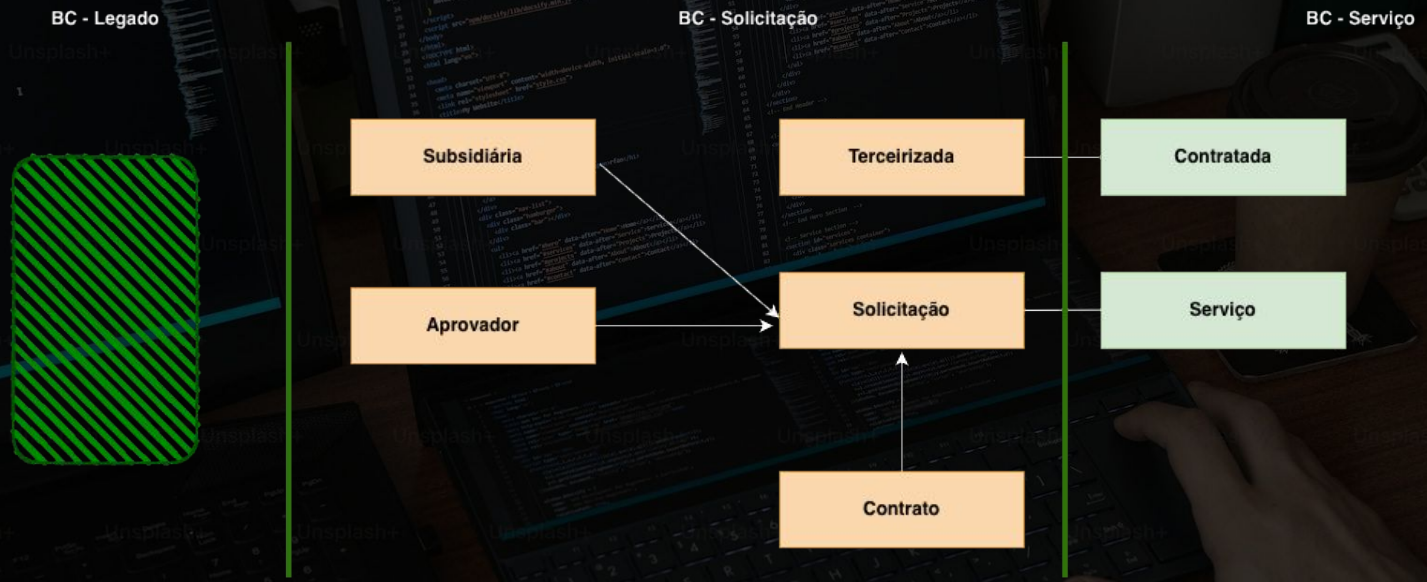
Geografia…

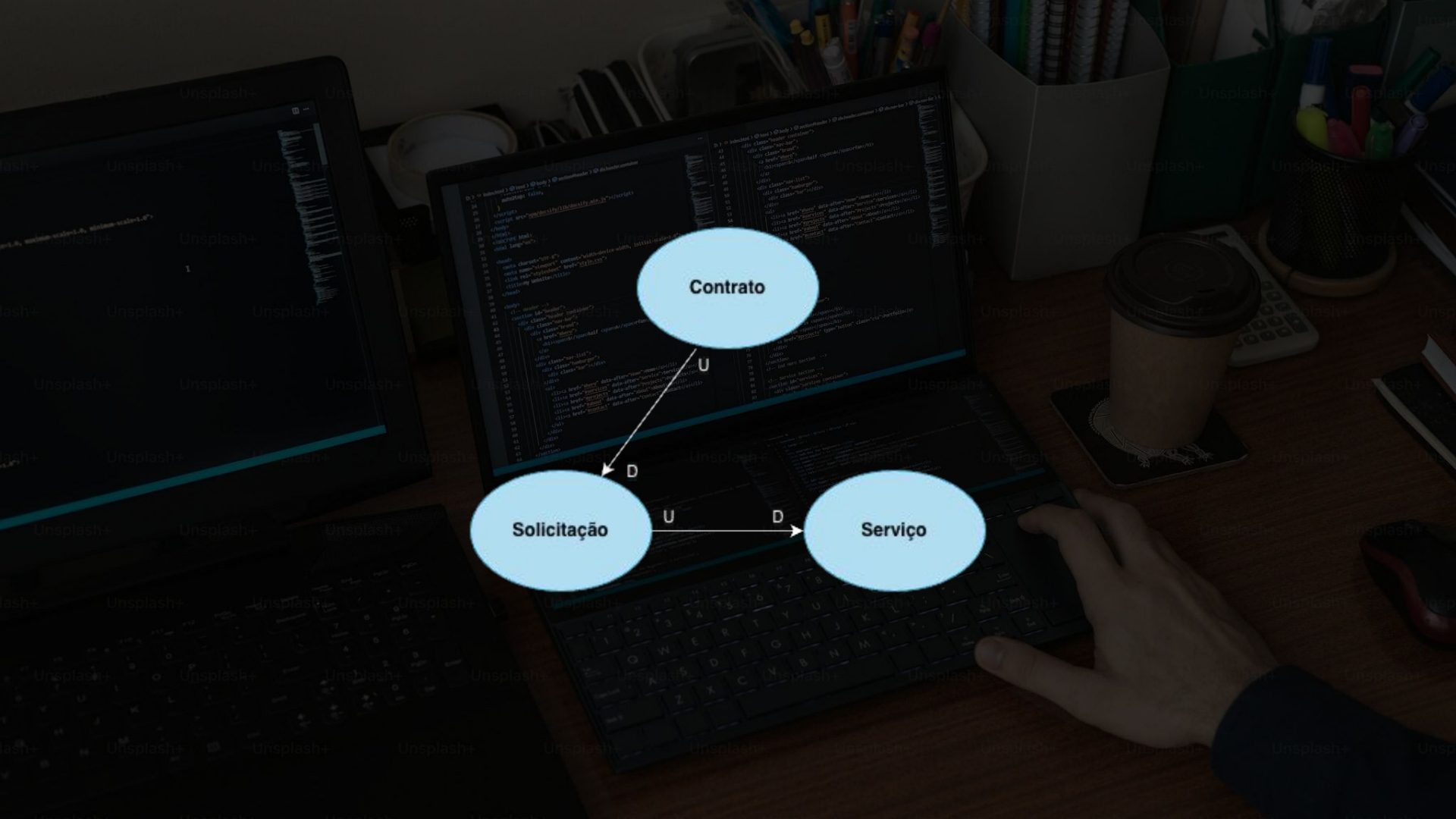
- Analista e Diretor Geral ficam na sede
- Gestor local fica na subsidiária

Isso ajuda a entender o porque um fala um termo e o outro fala outro termo para a mesma coisa…

Serviço = Solicitação

Terceirizada = Contratada


BC - Legado
BC - Solicitação
BC - Serviço
Subsidiária
Aprovador
Terceirizada
Solicitação
Contrato
Contratada
Serviço


Contrato
U
D
Solicitação
U
D
Serviço